

```
" <|-- "" Deployment : A release artifact can be deployed
Deployment "1" --> "1" Environment : Every deployment lives in a specific environment
```

See Glossary for more information.

Discussion: It's TBD whether we should reuse existing entities such as Deployment and Environment models. Reusing the existing entities could limit us in the long run, however, users should be able to adopt the new architecture seamlessly without changing their existing CI/CD workflow drastically. This decision should be made when we have clearer answer for the ideal structure and behavior of the entities, so that we can understand how far from it the existing entity is and how feasible to migrate.

Alternative Solutions

- Add dynamically populated organization-level environments page.
This approach was concluded as no-go in favor of Service concept.
- There is an alternative proposal to introduce Group Environment entity for Group-level environment views.

```
## SECTION: engineering/architecture/design-documents/gitlab_steps/_index.md
```

```
{{< engineering/design-document-header >}}
```

Summary

This document describes architecture of a new component called Step Runner, the GitLab Steps syntax it uses, and how the GitHub Actions support will be achieved.

The competitive CI products drone.io, GitHub Actions have a composable CI jobs execution in form of steps, or actions.

Their usage and our prior evaluation of GitLab Runner Plugins shows a need for a better way to define CI job execution.

Glossary

- GitLab Steps: a name of GitLab CI feature to define and use reusable components within a single job execution context.
- Step Runner: a RFC implementation for GitLab Steps that provides compatibility with the GitHub Actions.
- GitHub Actions: similar to GitLab Steps, a reusable execution component used on GitHub.
- CI Catalog: a public or private component catalog that could be used to discover and use shared components.
- GitLab Rails: a main application responsible for pipeline execution, running on GitLab.com or on-premise installation.

Motivation

Even though the current `.gitlab-ci.yml` file is reasonably flexible, it easily becomes very complex when trying to support complex workflows. This complexity is represented with repetitive patterns, a purpose-specific syntax, or a complex sequence of commands to execute.

This is particularly challenging, because the `.gitlab-ci.yml` file is inflexible on more complex workflows that require fine-tuning or special behavior for the CI job execution. Its prescriptive approach how to handle Git cloning, when artifacts are downloaded, or how the shell script is being executed quite often results in the need to work around the system for pipelines that are not "standard" or when new features are requested.

This proves especially challenging when trying to add a new syntax to the `.gitlab-ci.yml` file to support a specific feature, like secure files or release: keyword. Adding these special features on a syntax level results in a more complex config, which is harder to maintain, and more complex to deal with technical debt when requirements change.

An example of the drone.io and the GitHub Actions shows that a lot of workflows do not have to be part of CI syntax. Instead, they can be provided in the form of reusable components that are configured in a generic way in the CI config, and later downloaded and executed according to inputs and parameters.

GitLab Steps is meant to fill that product-gap by following similar model to competitors and to some extent staying compatible with them. The GitLab Steps is meant to replace all purpose-specific syntax to handle specific features. By providing and using reusable components, that are build outside of `.gitlab-ci.yml`, that are versioned, and requested when needed this allows the customer much more flexibility, and allows us to iterate on a catalog much faster.

The reusable components that are part of a CI job execution could be used from a publicly hosted repository on GitLab.com, from on-premise repository of steps, or be fetched from local project.

Each CI job would define a list of steps: to execute, that would reference GitLab Steps or GitHub Actions. Those steps would be executed by the step runner directly in the context of the target environment. GitLab Runner would be responsible to be connection between GitLab.com (or on-premise installation) and Step Runner.

Goals

GitLab Steps:

- GitLab Steps defines a syntax and structure for GitLab specific Steps implementation.

- GitLab Steps are published in CI Catalog.
- GitLab Steps can be used across instances (federation).
- GitLab Steps do define inputs and outputs.
- GitLab Steps needs to explicitly request sensitive informations with expected permissions.
For example: secrets, variables, tokens.

GitLab Inc. managed repository of GitLab Steps:

- GitLab Inc. provides a repository of GitLab Steps that are a drop-in replacement for all current purpose-specific syntax: artifacts:, cache:, release:, etc.
- GitLab Inc. will provide a generic step to execute shell steps supporting various shells (bash, powershell).
- The usage of purpose-specific syntax might be eventually deprecated in favor of steps.

Step Runner:

- Step Runner is hosted in a separate project in <https://gitlab.com/gitlab-org>.
- Step Runner can be used to execute most of GitHub Actions.
- Step Runner is run as a process in a target environment.
- Step Runner can be used by user on their local machine to run steps of a specific CI job from locally stored .gitlab-ci.yml.
- Step Runner is external component to GitLab Runner, the GitLab Runner does provision environment, construct payload and pass execution to Step Runner.
- Step Runner is to replace all custom handling in GitLab Runner for clone, artifacts, caches, script and afterscript, and custom handling for all different shells (bash, powershell).
- Step Runner is responsible for parsing and compiling GitLab Steps and GitHub Actions.
- Step Runner is responsible for downloading, and managing repositories required by GitLab Steps and GitHub Actions.
- Step Runner does control and monitor execution flow of individual steps of execution.
- Step Runner is required to be executable from the command-line interface (CLI). It means that it can be configured either via config file, or environment file, or be able to read .gitlab-ci.yml.
- Step Runner can expose gRPC or other programmable interface to run config or get trace from.

Steps Execution:

- Each Step is defined by a single published or locally defined GitLab Step, or GitHub Action.
- Each Step is executed depending on conditions that are defined by that step.
- Each Step is executed with least amount of information exposed. Exposed informations to step are requested explicitly by the step. For example: only environment variables explicitly requested by the step will be passed to the step.
- Each Step is considered untrusted. It means that even though some steps are trusted, the whole CI job should be considered untrusted, since system cannot guarantee trust.
- Each Step describes its execution in a form of preconditions, versions used, and output produced.
This is meant to allow to sign steps execution for the purpose of creating reproducible builds.

Backward compatibility:

- All currently executable syntax (for example: before_script:, script:, artifacts:, cache:, etc.) should be convertible by GitLab (Rails)

Non-Goals

TBD

Proposal

Step Runner will be a new go binary which lives at <https://gitlab.com/gitlab-org/step-runner>. It will be able to accept a number of input formats which are compiled to a standard proto format.

Output will be a standard proto trace which will include details for debugging and reproducing the build.

Capabilities

- Read steps
 - from environment variable
 - from .gitlab-ci.yml file
 - from gRPC server in step-runner
 - from commandline JSON input
- Compile GitLab Steps and GitHub Actions to a baseline step definition
 - explicit inputs and outputs
 - explicit environment and exports
 - baseline steps can be type exec or more steps
- Download and run steps from:
 - Git repos
 - zip files
 - locally provided
- A job can be composed of different kinds of steps
 - steps can come from different sources and be run in different ways
 - steps can access environment exports and output of previous steps
- Produce a step-by-step trace of execution
 - including final inputs and outputs
 - including final environment and exports
 - including logs of each step
 - each step specifies the exact runtime and component used (hash)
 - (optional) masking sensitive inputs, outputs, environment and exports
- Replaying a trace
 - reuses the exact runtimes and components from trace
 - output of trace will be the same trace if build is deterministic

Example invocations

Command line

- STEPS=\$(cat steps.yml) step-runner ci
- step-runner local .gitlab-ci.yml --format gitlab-ci --job-name hello-world --output-file trace.json
- step-runner replay trace.json
- step-runner ci --port 8080

GitLab CI

yaml

hello-world:

image: registry.gitlab.com/gitlab-org/step-runner

variables:

STEPS: |

- step: gitlab.com/josephburnett/component-hello-steppy@master

inputs:

greeting: "hello \${ env.name }"

env:

name: world

script:

- /step-runner ci

artifacts:

paths:

- trace.json

Basic compilation and execution process

Steps as expressed in GitLab CI are compiled to the baseline step definition. Referenced steps are loaded and compiled to produce an exec command, or to produce an additional list of GitLab CI steps which are compiled recursively. Each step is executed immediately after compilation so its output will be available for subsequent compilations.

Steps return outputs and exports via files which are collected by Step Runner after each step. Finally all the compiled inputs and outputs for each step are collected in a step trace.

GitLab Steps definition and syntax

- Step Definition.
- Syntactic Sugar extensions.

Integration of GitLab Steps

- Usage of the GitLab Steps with .gitlab-ci.yml.
- Runner Integration.

Design and implementation details

2023-11-28 - GitLab Steps ADR 001: Bootstrap Step Runner

- See the GitLab Steps ADR 001: Bootstrap Step Runner.
- See the Baseline Step Proto.

References

- GitLab Issue 215511
- Step Runner Code.
This is the exploratory code created during the writing of this blueprint.
It shows the structure of the Step Runner binary and how the pieces fit together.
It runs but doesn't quite do the right thing (see all the TODOs).
- CI Steps / CI Events / Executors / Taskonaut (video).
Some high-level discussion about how these 4 blueprints relate to each other.
And a good prequel to the video about this MR.
- Steps in Runner (video).
A walk through of the Step Runner details from the code perspective.
- CI YAML keywords.
An inventory of affected keywords.
- GitLab Epic 11535

SECTION: engineering/architecture/design-documents/gitlab_steps/gitlab-ci.md

This document describes how GitLab Steps are integrated into the .gitlab-ci.yml.

GitLab Steps will be integrated using a three-stage execution cycle and replace beforescript:, script: and afterscript:.

- setup:: Execution stage responsible for provisioning the environment, including cloning the repository, restoring artifacts, or installing all dependencies. This stage will replace implicitly cloning, restoring artifacts, and cache download.
- run:: Execution stage responsible for running a test, build, or any other main command required by that job.
- teardown:: Execution stage responsible for cleaning the environment, uploading artifacts, or storing cache. This stage will replace implicit artifacts and cache uploads.

Before we can achieve three-stage execution we will ship minimal initial support that does not require any prior GitLab integration.

Phase 1: Initial support

Initially the Step Runner will be used externally, without any prior dependencies to GitLab:

- The step-runner will be provided as part of a container image.

- The step-runner will be explicitly run in the script: section.
- The \$STEPS environment variable will be executed as type: steps.

```
yaml
hello-world:
  image: registry.gitlab.com/gitlab-org/step-runner
  variables:
    STEPS: |
      - step: gitlab.com/josephburnett/component-hello-steppy@master
  inputs:
    greeting: "hello world"
  script:
    - /step-runner ci
```

Phase 2: The addition of run: to .gitlab-ci.yml

In Phase 2 we will add run: as a first class way to use GitLab Steps:

- run: will use a type: steps syntax.
- run: will replace usage of before_script, script and after_script.
- All existing functions to support Git cloning, artifacts, and cache would continue to be supported.
- It is yet to be defined how we would support after_script, which is executed unconditionally or when the job is canceled.
- run: will not be allowed to be combined with before_script:, script: or after_script:.
- GitLab Rails would not parse run:, instead it would only perform static validation with a JSON schema provided by the Step Runner.

```
yaml
hello-world:
  image: registry.gitlab.com/gitlab-org/step-runner
  run:
    - step: gitlab.com/josephburnett/component-hello-steppy@master
  inputs:
    greeting: "hello world"
```

The following example would fail syntax validation:

```
yaml
hello-world:
  image: registry.gitlab.com/gitlab-org/step-runner
  run:
    - step: gitlab.com/josephburnett/component-hello-steppy@master
  inputs:
    greeting: "hello world"
  script: echo "This is ambiguous and invalid example"
```

Transitioning from beforescript:, script: and afterscript:

GitLab Rails would automatically convert the script: syntax into relevant run: specification:

- Today beforescript: and script: are joined together as a single script for execution.
- The afterscript: section is always executed in a separate context, representing a separate step to be executed.
- It is yet to be defined how we would retain the existing behavior of afterscript, which is always executed regardless of the job status or timeout, and uses a separate timeout.
- We would retain all implicit behavior which defines all environment variables when translating script: into step-based execution.

For example, this CI/CD configuration:

```
yaml
hello-world:
  beforescript:
    - echo "Run beforescript"
  script:
    - echo "Run script"
  afterscript:
    - echo "Run afterscript"
```

Could be translated into this equivalent specification:

```
yaml
hello-world:
  run:
    - step: gitlab.com/gitlab-org/components/steps/legacy/script@v1.0
      inputs:
        script:
          - echo "Run beforescript"
          - echo "Run script"
    - step: gitlab.com/gitlab-org/components/steps/legacy/script@v1.0
      inputs:
        script:
          - echo "Run afterscript"
  when: always
```

Phase 3: The addition of setup: and teardown: to .gitlab-ci.yml

The addition of setup: and teardown: will replace the implicit functions provided by GitLab Runner: Git clone, artifacts and cache handling:

- The usage of setup: would stop GitLab Runner from implicitly cloning the repository.
- artifacts: and cache:, when specified, would be translated and appended to setup: and

teardown:

to provide backward compatibility for the old syntax.

- release:, when specified, would be translated and appended to teardown: to provide backward compatibility for the old syntax.
- setup: and teardown: could be used in default: to simplify support of common workflows like where the repository is cloned, or how the artifacts are handled.
- The split into 3-stage execution additionally improves composability of steps with extends:.
- The hooks:pregetsourcesscript would be implemented similar to script: and be prepended to setup:.

For example, this CI/CD configuration:

yaml

rspec:

script:

- echo "This job uses a cache."

artifacts:

paths: [binaries/, .config]

cache:

key: binaries-cache

paths: [binaries/.apk, .config]

Could be translated into this equivalent specification executed by a step runner:

yaml

rspec:

setup:

- step: gitlab.com/gitlab-org/components/git/clone@v1.0
- step: gitlab.com/gitlab-org/components/artifacts/download@v1.0
- step: gitlab.com/gitlab-org/components/cache/restore@v1.0

inputs:

key: binaries-cache

run:

- step: gitlab.com/gitlab-org/components/steps/legacy/script@v1.0

inputs:

script:

- echo "This job uses a cache."

teardown:

- step: gitlab.com/gitlab-org/components/artifacts/upload@v1.0

inputs:

paths: [binaries/, .config]

- step: gitlab.com/gitlab-org/components/cache/restore@v1.0

inputs:

key: binaries-cache

paths: [binaries/.apk, .config]

Inheriting common operations with default:

setup: and teardown: are likely to become very verbose over time. One way to simplify them is to allow inheriting the common setup: and teardown: operations with default:.

The previous example could be simplified to:

yaml

default:

setup:

- step: gitlab.com/gitlab-org/components/git/clone@v1.0
- step: gitlab.com/gitlab-org/components/artifacts/download@v1.0
- step: gitlab.com/gitlab-org/components/cache/restore@v1.0

inputs:

key: binaries-cache

teardown:

- step: gitlab.com/gitlab-org/components/artifacts/upload@v1.0

inputs:

paths: [binaries/, .config]

- step: gitlab.com/gitlab-org/components/cache/restore@v1.0

inputs:

key: binaries-cache

paths: [binaries/.apk, .config]

rspec:

run:

- step: gitlab.com/gitlab-org/components/steps/legacy/script@v1.0

inputs:

script:

- echo "This job uses a cache."

linter:

run:

- step: gitlab.com/gitlab-org/components/steps/legacy/script@v1.0

inputs:

script:

- echo "Run linting"

Parallel jobs and setup:

With the introduction of setup: at some point in the future we will introduce an efficient way to parallelize the jobs:

- setup: would define all steps required to provision the environment.
- The result of setup: would be snapshot and distributed as the base for all parallel jobs, if parallel: N is used.
- The run: and teardown: would be run on top of cloned job, and all its services.
- The runner would control and intelligently distribute all parallel jobs, significantly cutting the resource requirements for fixed parts of the job (Git clone, artifacts, installing dependencies.)

```

yaml
rspec-parallel:
  image: ruby:3.2
  services: [postgres, redis]
  parallel: 10
  setup:
    - step: gitlab.com/gitlab-org/components/git/clone@v1.0
    - step: gitlab.com/gitlab-org/components/artifacts/download@v1.0
    inputs:
      jobs: [setup-all]
    - script: bundle install --without production
  run:
    - script: bundle exec knapsack

```

Potential GitLab Runner flow:

1. Runner receives the rspec-parallel job with setup: and parallel: configured.
1. Runner executes a job on top of Kubernetes cluster using block volumes up to the setup.
1. Runner then runs 10 parallel jobs in Kubernetes, overlaying the block volume from 2 and continue execution of run: and teardown:.

SECTION: engineering/architecture/design-documents/gitlab_steps/implementation.md

Baseline Step Proto

The internals of Step Runner operate on the baseline step definition which is defined in Protocol Buffer. All GitLab CI steps (and other supported formats such as GitHub Actions) compile / fold to baseline steps. Both step invocations in .gitlab-ci.yml and step definitions in step.yml files will be compiled to baseline structures. The term "step" means "baseline step" for the remainder of this document.

Each step includes a reference ref in the form of a URI. The method of retrieval is determined by the protocol of the URI.

Steps and step traces have fields for inputs, outputs, environment variables and environment exports. After steps are downloaded and the step.yml is parsed a step definition def will be added. If a step defines multiple additional steps then the trace will include sub-traces for each sub-step.

```

protobuf
message Step {
  string name = 1;
  string step = 2;
  map<string,string> env = 3;

```

```
    map<string,google.protobuf.Value> inputs = 4;
}
```

```
message Definition {
    DefinitionType type = 1;
    Exec exec = 2;
    repeated Step steps = 3;
    message Exec {
        repeated string command = 1;
        string workdir = 2;
    }
}
```

```
enum DefinitionType {
    definitiontypeunspecified = 0;
    exec = 1;
    steps = 2;
}
```

```
message Spec {
    Content spec = 1;
    message Content {
        map<string,Input> inputs = 1;
        message Input {
            InputType type = 1;
            google.protobuf.Value default = 2;
        }
    }
}
```

```
enum InputType {
    spectypeunspecified = 0;
    string = 1;
    number = 2;
    bool = 3;
    struct = 4;
    list = 5;
}
```

```
message StepResult {
    Step step = 1;
    Spec spec = 2;
    Definition def = 3;
    enum Status {
        unspecified = 0;
        running = 1;
        success = 2;
        failure = 3;
    }
    Status status = 4;
```

```

map<string,Output> outputs = 5;
message Output {
  string key = 1;
  string value = 2;
  bool masked = 3;
}
map<string,string> exports = 6;
int32 exitcode = 7;
repeated StepResult childrenstepresults = 8;
}

```

Step Caching

Steps are cached locally by a key comprised of location (URL), version and hash. This prevents the exact same component from being downloaded multiple times. The first time a step is referenced it will be downloaded (unless local) and the cache will return the path to the folder containing step.yml and the other step files. If the same step is referenced again, the same folder will be returned without downloading.

If a step is referenced which differs by version or hash from another cached step, it will be re-downloaded into a different folder and cached separately.

Execution Context

State is kept by Step Runner across all steps in the form of an execution context. The context contains the output of each step, environment variables and overall job and environment metadata. The execution context can be referenced by expressions in GitLab CI steps provided by the workflow author.

Example of context available to expressions in .gitlab-ci.yml:

```

yaml
steps:
  previousstep:
    outputs:
      name: "hello world"
env:
  EXAMPLEVAR: "bar"
job:
  id: 1234

```

Expressions in step definitions can also reference execution context. However they can only access overall job and environment metadata and the inputs defined in step.yml.

They cannot access the outputs of previous steps. In order to provide the output of one step to the next, the step input values should include an expression which references another step's output.

Example of context available to expressions in step.yml:

```
yaml
inputs:
  name: "foo"
env:
  EXAMPLEVAR: "bar"
job:
  id: 1234
```

E.g. this is not allowed in a step.yml file because steps should not couple to one another.

```
yaml
spec:
  inputs:
    name:

type: exec
exec:
  command: [echo, hello, ${{ inputs.name }}]
```

```
yaml
steps:
- name: previousstep
  ...
- name: greeting
  inputs:
    name: ${{ steps.previousstep.outputs.name }}
```

Therefore evaluation of expressions will be done in two different kinds of context. One as a GitLab CI Step and one as a step definition.

Step Inputs

Step inputs can be given in several ways. They can be embedded directly into expressions in an exec command (as above). Or they can be embedded in expressions for environment variables set during exec:

```
yaml
spec:
```

```
inputs:
  name:

type: exec
exec:
  command: [echo, ${{ inputs.foo }}, ${{ inputs.baz }}, ${{ inputs.bam }}]
```

And it would output bar true 1

For an object type, these would be valid inputs:

```
yaml
steps:
  name: mystep
  inputs:
    foo:
      steps:
        - name: myinnerstep
          inputs:
            name: steppy
```

Given this step definition:

```
yaml
spec:
  inputs:
    foo:
      type: object
---
type: exec
exec:
  command: [echo, ${{ inputs.foo }}]
```

And it would output {"steps":[{"name":"myinnerstep","inputs":{"name":"steppy"}}]}

Outputs

Output files are created into which steps can write their outputs and environment variable exports. The file locations are provided in OUTPUTFILE and ENVFILE environment variables.

After execution Step Runner will read the output and environment variable files and populate the trace with their values. The outputs will be stored under the context for the executed step. And the exported environment variables will be merged with environment provided to the next step.

Some steps can be of type steps and be composed of a sequence of GitLab CI steps. These will be compiled and executed in sequence. Any environment variables exported by nested steps will be available to subsequent steps. And will be available to high level steps when the nested steps are complete. E.g. entering nested steps does not create a new "scope" or context object. Environment variables are global.

Containers

We've tried a couple approaches to running steps in containers. In end we've decided to delegate steps entirely to a step runner in the container.

Here are the options considered:

Delegation (chosen option)

A provision is made for passing complex structures to steps, which is to serialize them as JSON (see Inputs above). In this way the actual step to be run can be merely a parameter to step running in container. So the outer step is a docker/run step with a command that executes step-runner with a steps input parameter. The docker/run step will run the container and then extract the output files from the container and re-emit them to the outer steps.

This same technique will work for running steps in VMs or whatever. Step Runner doesn't have to know anything about containerizing or isolation steps.

Special Compilation (rejected option)

When we see the image keyword in a GitLab CI step we would download and compile the "target" step. Then manufacture a docker/run step and pass the compiled exec command as an input. Then we would compile the docker/run step and execute it.

However this requires Step Runner to know how to construct a docker/run step. Which couples Step Runner with the method of isolation, making isolation in VMs and other methods more complicated.

Native Docker (rejected option)

The baseline step can include provisions for running a step in a Docker container. For example the step could include a ref "target" field and an image field.

However this also couples Step Runner with Docker and expands the role of Step Runner. It is preferable to make Docker an external step that Step Runner execs in the same way as any other step.

SECTION: engineering/architecture/design-documents/gitlab_steps/runner-integration.md

Non goals

This proposal does not address deployment of the Step Runner binary into target environments, nor of starting the Step Runner gRPC service described below. The rest of the proposal assumes both that the Step Runner binary exists in the target environment and that the gRPC service is running and listening on a local socket. Similarly this proposal does not address the life-cycle of the Step Runner service, and how to handle things like restarting the service if it dies, or upgrades.

See Deployment and Lifecycle Management for relevant blueprint.

Steps Service gRPC Definition

The Step Runner service gRPC definition is as follows:

```
proto
service StepRunner {
  rpc Run(RunRequest) returns (RunResponse);
  rpc FollowSteps(FollowStepsRequest) returns (stream FollowStepsResponse);
  rpc FollowLogs(FollowLogsRequest) returns (stream FollowLogsResponse);
  rpc Finish(FinishRequest) returns (FinishResponse);
  rpc Status(StatusRequest) returns (StatusResponse);
}

message Variable {
  string key = 1;
  string value = 2;
  bool file = 3;
  bool masked = 4;
}

message Job {
  repeated Variable variables = 1;
  string jobid = 2;
  string pipelineid = 3;
  string builddir = 4;
  repeated string tokenprefixes = 5;
}

message Masking {
  repeated string phrases = 1;
  repeated string tokenprefixes = 2;
}

message RunRequest {
```

```

    string id = 1;
    string workdir = 2;
    map<string,string> env = 3;
    Masking masking = 4;
    Job job = 5;
    string steps = 6;
}

message RunResponse {
}

message FollowStepsRequest {
    string id = 1;
}

message FollowStepsResponse {
    StepResult result = 1;
}

message FollowLogsRequest {
    string id = 1;
    int32 offset = 2;
}

message FollowLogsResponse {
    bytes data = 1;
}

message FinishRequest {
    string id = 1;
}

message FinishResponse {
}

message Status {
    string id = 1;
    bool finished = 2;
    int32 exitcode = 3;
    google.protobuf.Timestamp starttime = 4;
    google.protobuf.Timestamp endtime = 5;
}

message StatusRequest {
    string id = 1;
}

message StatusResponse {
    repeated Status jobs = 1;
}

```

Runner interacts with Step Runner over the above gRPC service which is started on a local socket in the execution environment. Runner accesses the local socket by first connecting to the target environment via executor-specific protocols, then use a provided proxy command to connect to the gRPC service, and transparently tunnel gRPC requests from the Runner to Step Runner (seeProxy Command). This is the same way that Nesting serves a gRPC service in a dedicated Mac instance. The service has five RPCs, Run, FollowSteps, FollowLogs, Finish and Status.

Run is the initial delivery of the steps. FollowSteps requests a streaming response of step-result traces. FollowLogs similarly requests a streaming response of output (stdout/stderr) written by processes executed as part of running the steps, and logs produced by Step Runner itself. Finish stops execution of the request (if still running) and cleans up resources as soon as possible. Status lists the status of the specified job, or if no job was specified, of all active jobs in the Step Runner service (including completed but not Finished jobs). Status can for example be used by a runner to recover after a crash.

The Step Runner gRPC service will be able to execute multiple Run payloads at once. That is, each call to Run will start a new goroutine and execute the steps until completion. Multiple calls to Run may be made simultaneously.

As steps are executed, step-result traces and sub-process logs can be streamed back to GitLab Runner. This allows Runner (or any caller) to follow execution, at the step level for step-result traces (FollowSteps), and as written for sub-process and Step Runner logs (FollowLogs). Logs will be written in a specific format, and sensitive tokens will be masked by Step Runner before being streamed to Runner.

All APIs excluding Status are idempotent, meaning that multiple calls to the same API with the same parameters should return the same result. For example, If Run is called multiple times with the same id, only the first invocation should begin processing of the job request, and subsequent invocations return a success status but otherwise do nothing. Similarly, multiple calls to Finish with the same id should finish and remove the relevant job on the first call, and do nothing on subsequent calls.

The service should not assume clients will be well-behaved, and should be able to handle clients that never call or prematurely disconnect from either of the Follow APIs, and also clients that never call Finish on a corresponding Run request. To this end the Step Runner process should periodically perform a scan to identify and prune stale or runaway/stuck jobs. A stale job could be a job that has finished some specified time ago (and has not been Finished). A runaway job is a job

that has been running some (long) specified amount of time, possibly without producing output.

Finally, to facilitate integrating steps into the below runner executors, it is recommended that steps provide a client library to coordinate execution of the Run/Follow/Finish APIs, and to handle reconnecting to the step-runner service in the event that the Follow calls loose connectivity.

RunRequest Parameters

Steps are delivered to Step Runner in the RunRequest.Steps field as a JSON-serialized version of step.go, with no processing of the step definition required by runner itself. The id field uniquely identifies each request running on the Step Runner service. The RunRequest.Env field holds environment variable that are to be injected into the environment when each step is executed.

The optional Job parameter will include select parameters from the corresponding CI job. Job will include the corresponding CI job's build directory; Job.BuildDir should be copied to RunRequest.WorkDir, and all steps in a request should be invoked in that directory to preserve existing job script behavior. The RunRequest will also include the CI job's environment variables (i.e. the variables defined at the job and global levels in the CI configuration). When a RunRequest is made by Runner, variables must be included in Job.Variables, and RunRequest.Env should be left empty. When the run request is processed, file-type variables will be written to file, variables will be expanded, copied into RunRequest.Env, and the Job field will be discarded from the remainder of the request. Variables should be expanded by the Step Runner service since they may reference object in the execution environment (like other environment variables or paths). This includes file-type variables, which should be written to the same path as they are be in traditional runner job execution. Similarly, from Job.Variables, phrases to be masked should be extracted and used to populate Masking.Phrases, and Job.TokenPrefixes should be copied into Masking.TokenPrefixes.

Clients other than Runner wishing to run steps can omit the Job field, and in this case the Masking and Env fields should be populated directly by the caller.

Log Format

Log lines emitted buy the FollowLogs API should have the format

plaintext

```
<timestamp> <stream> <stdout/stderr> <append flag> <message>
```

This is the same log format introduced into runner in this merge request. The logging library used to produce this format should be shared between GitLab Runner and Step Runner.

Masking

Step Runner will be responsible for masking sensitive variables or tokens. This should be done before the raw log message is formatted into the above log format. The libraries used to mask variables should be shared between GitLab Runner and Step Runner. (See relevant modules).

Proxy Command

The Step Runner binary will include a command to proxy data from (typically text-based) stdin/stdout/stderr-based protocols to the gRPC service. This command will run in the same host as the gRPC service, and will read input from stdin, forward it to the gRPC service over a local socket, receive output from the gRPC service over same socket, and forward it to the client via stdout/stderr. This command will enable clients (like Runner) to transparently tunnel to the gRPC service via stdin/stderr/stdout-based protocols like SSH or docker exec, which will eliminate the need to expose the Step Runner service's gRPC port on Docker images, or set up SSH port forwarding on VMs, and will allow runner to interact with Step Runner using established protocols (i.e. SSH and docker exec). stdout should be reserved for writing responses from the Step Runner service, and stderr should be reserved for errors originating in the proxy command itself.

Executors

Here is how GitLab Runner will connect to Step Runner in each runner executor:

Instance

The Instance executor is accessed via SSH, the same as today. However instead of starting a bash shell and piping in commands, it invokes the proxy command, which in turn connects to the Step Runner socket in a known location. Runner can then make gRPC calls directly, and transparently tunnel through the SSH connection to the gRPC service. This is the same as how Runner calls the Nesting server in dedicated Mac instances to make VMs.

This requires that Step Runner is present and started in the job execution environment.

Docker

The same requirement that Step Runner is present and the gRPC service is running is true for the Docker executor (and docker-autoscaler). However in order to connect to the gRPC service inside the container, Runner will docker exec to the container and execute the proxy command to connect to the gRPC service in the container. The client can then write to the docker exec's stdin, which will transparently be proxied to the gRPC service, and read from its stdout/stderr, which will contain responses from the gRPC service.

Kubernetes

The Kubelet on Kubernetes Nodes exposes an exec API which will start a process in a container of a running Pod. We will use this to exec create a bridge process that will allow the caller to make gRPC calls inside the Pod, similar to the Docker executor.

In order to access to this protected Kubelet API we must use the Kubernetes API which provides an exec sub-resource on Pod. A caller can POST to the URL of a pod suffixed with /exec and then negotiate the connection up to a SPDY protocol for bidirectional byte streaming. So GitLab Runner can use the Kubernetes API to connect to the Step Runner service and deliver job payloads.

This is the same way that kubectl exec works. In fact most of the internals such as SPDY negotiation are provided as client-go libraries. So Runner can call the Kubernetes API directly by importing the necessary libraries rather than shelling out to Kubectl.

Historically one of the weaknesses of the Kubernetes Executor was running a whole job through a single exec. To mitigate this Runner uses the attach command instead, which can "re-attach" to an existing shell process and pick up where it left off.

This is not necessary for Step Runner however, because the exec is just establishing a bridge to the long-running gRPC process. If the connection drops, Runner will just "re-attach" by exec'ing another connection and continuing to make RPC calls like follow.

SECTION: engineering/architecture/design-documents/gitlab_steps/service-deployment.md

This Blueprint is concerned with:

- The deployment or injection of the Step Runner binary into target environments. This includes build containers for Docker, Kubernetes and Instance executors.
- Startup of the Step Runner gRPC service in said environments.
- Any required install-time configuration.

- Service restart in the event of a crash.
- Step Runner binary upgrade for environments where the Step Runner service is long lived.
- Management of any resources used by the Step Runner service

SECTION: engineering/architecture/design-documents/gitlab_steps/step-definition.md

A step is the minimum executable unit that user can provide and is defined in a step.yml file.

The following step definition describes the minimal syntax supported.
The syntax is extended with syntactic sugar.

A step definition consists of two documents. The purpose of the document split is to distinguish between the declaration and implementation:

1. Specification / Declaration:

Provides the specification which describes step inputs and outputs, as well any other metadata that might be needed by the step in the future (license, author, etc.).

In programming language terms, this is similar to a function declaration with arguments and return values.

1. Implementation:

The implementation part of the document describes how to execute the step, including how the environment

has to be configured, or how actions can be configured.

Example step that prints a message to stdout

In the following step example:

1. The declaration specifies that the step accepts a single input named message.

The message is a required argument that needs to be provided when running the step because it does not define default:.

1. The implementation section specifies that the step is of type exec. When run, the step will execute an echo command with a single argument (the message value).

```
yaml
.gitlab/ci/steps/exec-echo.yaml
```

```
spec:
  inputs:
    message:
```

```
type: steps
steps:
  - step: ./bash-script.yaml
    inputs:
```

```
script: "echo Code Coverage = 95.4% >> $OUTPUTFILE"
```

Step Implementation

The step definition can use the following types to implement the step:

- type: exec: Run a binary command, using STDOUT/STDERR for tracing the executed process.
- type: steps: Run a sequence of steps.
- type: parallel (Planned): Run all steps in parallel, waiting for all of them to finish.
- type: grpc (Planned): Run a binary command but use gRPC for intra-process communication.
- type: container (Planned): Run a nested Step Runner in a container image of choice, transferring all execution flow.

The exec step type

The ability to run binary commands is one of the primitive functions:

- The command to execute is defined by the exec: section.
- The result of the execution is the exit code of the command to be executed, unless the default behavior is overwritten.
- The default working directory in which the command is executed is the directory in which the step is located.
- By default, the command is not time-limited, but can be time-limited during job execution with timeout:.

For example, an exec step with no inputs:

yaml

spec:

type: exec

exec:

command: [/usr/bin/env, bash, -c, "set -veo pipefail; \${inputs.script}"]

The steps step type

The ability to run multiple steps in sequence is one of the primitive functions:

- A sequence of steps is defined by an array of step references: steps: [].
- The next step is run only if previous step succeeded, unless the default behavior is overwritten.
- The result of the execution is either:
 - A failure at the first failed step.
 - Success if all steps in sequence succeed.

Steps that use other steps

The steps type depends extensively on being able to use other steps.

Each item in a sequence can reference other external steps, for example:

```
yaml
spec:

type: steps
env:
  HTTPTIMEOUT: ${inputs.httptimeout}
steps:
  - step: ./gitlab/ci/steps/exec-echo.yaml
    inputs:
      message: "Installing Ruby ${inputs.rubyversion}..."
  - step: ./gitlab/ci/ruby/install.yaml
    inputs:
      version: ${inputs.rubyversion}
```

Context and interpolation

Every step definition is executed in a context object which stores the following information that can be used by the step definition:

- inputs: The list of inputs, including user-provided or default.
- outputs: The list of expected outputs.
- env: The current environment variable values.
- job: The metadata about the current job being executed.
 - job.project: Information about the project, for example ID, name, or full path.
 - job.variables: All CI/CD Variables as provided by the CI/CD execution, including project variables, predefined variables, etc.
 - job.pipeline: Information about the current executed pipeline, like the ID, name, full path
- step: Information about the current executed step, like the location of the step, the version used, or the specification.
- steps (only for type: exec):
 - Information about each step in sequence to be run, containing information about the result of the step execution, like status or trace log.
 - steps.<name-of-the-step>.status: The status of the step, like success or failed.
 - steps.<name-of-the-step>.outputs.<output-name>: To fetch the output provided by the step

The context object is used to enable support for the interpolation in the form of `${<value>}`.

Interpolation:

- Is forbidden in the step specification section.

The specification is static configuration that should not be affected by the runtime environment.
- Can be used in the step implementation section. The implementation describes the runtime set of instructions for how the step should be executed.
- Is applied to every value of the hash of each data structure.
- Of the values of each hash is possible (for now). The interpolation of keys is forbidden.

- Is done when executing and passing control to a given step, instead of running it once when the configuration is loaded. This enables chaining outputs to inputs, or making steps depend on the execution of earlier steps.

For example:

```
yaml
.gitlab/ci/steps/exec-echo.yaml
spec:
  inputs:
    timeout:
      default: 10s
    bashsupportversion:
---
type: steps
env:
  HTTPTIMEOUT: ${inputs.timeout}
  PROJECTID: ${job.project.id}
steps:
- step: ./my/local/step/to/echo.yml
  inputs:
    message: "I'm currently building a project: ${job.project.fullpath}"
- step: gitlab.com/gitlab-org/components/bash/script@v${inputs.bashsupportversion}
```

Reference data structures describing YAML document

```
go
package main
```

```
type StepEnvironment map[string]string
```

```
type StepSpecInput struct {
    Default string yaml:"default"
    Description string yaml:"description"
    Options []string yaml:"options"
    Match string yaml:"match"
}
```

```
type StepSpecOutput struct {
}
```

```
type StepSpecInputs map[string]StepSpecInput
type StepSpecOutputs map[string]StepSpecOutput
```

```
type StepSpec struct {
    Inputs StepSpecInput yaml:"inputs"
    Outputs StepSpecOutputs yaml:"outputs"
}
```

```

type StepSpecDoc struct {
    Spec StepSpec yaml:"spec"
}

type StepType string

const StepTypeExec StepType = "exec"
const StepTypeSteps StepType = "steps"

type StepDefinition struct {
    Def StepSpecDoc      yaml:"- "
    Env StepEnvironment   yaml:"env"
    Steps StepDefinitionSequence yaml:"steps"
    Exec StepDefinitionExec  yaml:"exec"
}

type StepDefinitionExec struct {
    Command []string    yaml:"command"
    WorkingDir string     yaml:"workingdir"
    Timeout time.Duration yaml:"timeout"
}

type StepDefinitionSequence []StepReference

type StepReferenceInputs map[string]string

type StepReference struct {
    Step string      yaml:"step"
    Inputs StepReferenceInputs yaml:"inputs"
    Env StepEnvironment  yaml:"env"
}

```

SECTION: engineering/architecture/design-documents/gitlab_steps/steps-syntactic-sugar.md

The Step Definition describes a minimal required syntax to be supported. To aid common workflows the following syntactic sugar is used to extend different parts of that document.

Syntactic Sugar for Step Reference

Each of syntactic sugar extensions is converted into the simple step reference.

Easily execute scripts in a target environment

script: is a shorthand syntax to aid execution of simple scripts, which cannot be used with step:
and is run by an externally stored step component provided by GitLab.

The GitLab-provided step component performs shell auto-detection unless overwritten, similar to how GitLab Runner does that now: based on a running system.

inputs: and env: can be used for additional control of some aspects of that step component.

For example:

yaml
spec:

```
type: steps
steps:
  - step: gitlab.com/gitlab-org/components/steps/script@v1.0
    inputs:
      script: bundle exec rspec
  - step: gitlab.com/gitlab-org/components/steps/script@v1.0
    inputs:
      script: bundle exec rspec
      shell: sh Force runner to use sh shell, instead of performing auto-detection
```

This syntax example is invalid (and ambiguous) because the script: and step: cannot be used together:

```
yaml
spec:
---
type: steps
steps:
  - step: gitlab.com/my-component/ruby/install@v1.0
    script: bundle exec rspec

---
## SECTION: engineering/architecture/design-
documents/gitlab_steps/decisions/001_initial_support.md
```

Context

GitLab Steps is a new feature that does not have any prior usage at GitLab. We decided that there are two important objectives at this stage of the project:

- Integrate the project into existing CI pipelines for the purpose of user evaluation as part of an experiment phase.
- Provide a contribution framework for other developers in the form of a project with contribution guidelines.

Decision

The GitLab Steps: Iteration 1: Bootstrap Step Runner (MVC)

was created to achieve the following objectives:

- We defined the initial plan to bootstrap the project.
- The project will be stored in gitlab-org/step-runner.
- We will implement the Step Definition as a Protocol Buffer. The initial implementation is described in the Baseline Step Proto.
- Usage of Protocol Buffers will provide strong guards for the minimal required definition to be used by the project.
- We will provide documentation on how to use GitLab Steps in existing CI pipelines.

Alternatives

No alternatives were considered at this phase, since there's no pre-existing work at GitLab for that type of feature.

SECTION: engineering/architecture/design-
documents/gitlab_to_kubernetes_communication/_index.md

{{< engineering/design-document-header >}}

The goal of this document is to define how GitLab can communicate with Kubernetes and in-cluster services through the GitLab agent.

Challenges

Lack of network connectivity

For various features that exist today, GitLab communicates with Kubernetes by directly or indirectly calling its API endpoints. This works well, as long as a network path from GitLab to the cluster exists, which isn't always the case:

- GitLab.com and a self-managed cluster, where the cluster is not exposed to the Internet.
- GitLab.com and a cloud-vendor managed cluster, where the cluster is not exposed to the Internet.
- Self-managed GitLab and a cloud-vendor managed cluster, where the cluster is not exposed to the Internet and there is no private peering between the cloud network and the customer's network.

This last item is the hardest to address, as something must give to create a network path. This feature gives the customer an extra option (exposing the gitlab-kas domain but not the whole GitLab) in addition to the existing options (peering the networks, or exposing one of the two sides).

Even if technically possible, it's almost always undesirable to expose a Kubernetes cluster's API to the Internet for security reasons. As a result, our customers are reluctant to do so, and are faced with a choice of security versus the features GitLab provides for connected clusters.

This choice is true not only for Kubernetes' API, but for all APIs exposed by services running on a customer's cluster that GitLab may need to access. For example, Prometheus running in a cluster must be exposed for the GitLab integration to access it.

Cluster-admin permissions

Both current integrations - building your own cluster (certificate-based) and GitLab-managed cluster in a cloud - require granting full cluster-admin access to GitLab. Credentials are stored on the GitLab side and this is yet another security concern for our customers.

For more discussion on these issues, read [issue 212810](#).

GitLab agent epic

To address these challenges and provide some new features, the Configure group is building an active in-cluster component that inverts the direction of communication:

1. The customer installs an agent into their cluster.
1. The agent connects to GitLab.com or their self-managed GitLab instance, receiving commands from it.

The customer does not need to provide any credentials to GitLab, and is in full control of what permissions the agent has.

For more information, visit the [GitLab agent repository](#) or the epic.

Request routing

Agents connect to the server-side component called GitLab agent server (gitlab-kas) and keep an open connection that waits for commands. The difficulty with the approach is in routing requests from GitLab to the correct agent. Each cluster may contain multiple logical agents, and each may be running as multiple replicas (Pods), connected to an arbitrary gitlab-kas instance.

Existing and new features require real-time access to the APIs of the cluster and (optionally) APIs of components, running in the cluster. As a result, it's difficult to pass the information back and forth using the more traditional polling approach.

A good example to illustrate the real-time need is Prometheus integration. If we wanted to draw real-time graphs, we would need direct access to the Prometheus API to make queries and quickly return results. gitlab-kas could expose the Prometheus API to GitLab, and transparently route traffic to one of the correct agents connected at the moment. The agent then would stream the request to Prometheus and stream the response back.

Proposal

Implement request routing in gitlab-kas. Encapsulate and hide all related complexity from the main application by providing a clean API to work with Kubernetes and the agents.

The above does not necessarily mean proxying Kubernetes' API directly, but that is possible should we need it.

What APIs gitlab-kas provides depends on the features developed, but first we must solve the request routing problem. It blocks any and all features that require direct communication with agents, Kubernetes or in-cluster services.

Detailed implementation proposal with all technical details is in `kasrequestrouting.md`.

mermaid

flowchart LR

```
subgraph "Kubernetes 1"
    agentk1p1["agentk 1, Pod1"]
    agentk1p2["agentk 1, Pod2"]
end
```

```
subgraph "Kubernetes 2"
    agentk2p1["agentk 2, Pod1"]
end
```

```
subgraph "Kubernetes 3"
    agentk3p1["agentk 3, Pod1"]
end
```

```
subgraph kas
    kas1["kas 1"]
    kas2["kas 2"]
    kas3["kas 3"]
end
```

```
GitLab["GitLab Rails"]
Redis
```

```
GitLab -- "gRPC to any kas" --> kas
kas1 -- register connected agents --> Redis
kas2 -- register connected agents --> Redis
kas1 -- lookup agent --> Redis
```

```
agentk1p1 -- "gRPC" --> kas1
agentk1p2 -- "gRPC" --> kas2
agentk2p1 -- "gRPC" --> kas1
agentk3p1 -- "gRPC" --> kas2
```

Iterations

Iterations are tracked in the dedicated epic.

SECTION: engineering/architecture/design-documents/gitlab_translation_service/_index.md

{{< engineering/design-document-header >}}

Summary

This design document outlines the architectural design for an automated translation tool and a pipeline that provides translations for GitLab repositories and Translation Management Systems (TMS) used by Translation Vendors using a middleware Localization Request Management system called Argo. The integration tool is designed to automatically monitor and process translation requests for specified content changes in GitLab repositories through an event-driven Java application.

The objective of this design document is, however, not entirely on the application itself, but rather how we deploy this application in our GitLab infrastructure. But both of them will be discussed throughout the document.

The integration tool (which is a java application) implements an asynchronous, queue-based processing system that handles high-volume events from multiple projects/repositories while maintaining data consistency and preventing race conditions. It orchestrates the entire translation workflow - from detecting file changes in specific folders of interest, routing content through Argo's request management pipeline, all the way to managing the commit of translated content back to GitLab repositories.

The tool will be deployed in GitLab infrastructure:

1. The application code is hosted through a GitLab repository under Localization group, and builds the image file using GitLab's CI/CD pipeline
2. The relevant database is hosted through Google Cloud,
3. And deployed using Runway and connects to the GCP CloudSQL instance using PSC (Private Service Connect) and Cloud SQL Auth Proxy.

Motivation

The development of this automated localization pipeline addresses several critical challenges:

1. Manual overhead reduction:
 - Previously, all localization processes required manual intervention, creating a significant bottleneck,
 - Project managers were spending excessive time coordinating between development teams and translation vendors,
 - High risk of human error in file handling and version management.
2. Operational efficiency:

- Automates the complete workflow from source string detection to translation deployment,
- Eliminates the need for manual file transfers between systems,
- Significantly reduces the time from source string updates to localized content

availability.

3. Scalability benefits:

- Capable of handling increasing volumes of translation requests without additional resource allocation,
- Supports multiple repositories and projects simultaneously,
- Easily adaptable to new language requirements and vendor integrations.

4. Quality assurance:

- Maintains consistent file structure and naming conventions,
- Ensures version control integrity through automated commit processes,
- Reduces the risk of missing translations or outdated content.

5. Cost effectiveness:

- Reduces operational costs associated with manual localization management,
- Minimizes project delays and resource allocation issues,
- Optimizes translation vendor utilization through streamlined request management.

This automated pipeline represents a significant step forward in modernizing the localization workflow, ensuring both efficiency and accuracy in managing multilingual content across GitLab repositories.

Goals

Current Goals

Simplify and automate GitLab's translation pipeline with its translation vendors:

- Introduces an end-to-end automated localization infrastructure and pipelines - translating strings located in a GitLab repository,
- No manual human intervention required - engineers, PMs, and other associate stakeholders would save hours managing translations between GitLab and translation vendors,
- Ensuring higher accuracy in file handling and organizing large projects and decrease time-to-market for localized content,

Future Goals

- Ability to create translation requests from within Argo by selecting the files manually, and then automate to run on a schedule.
- Create a robust error handling and automated quality checks for file formats.
- Set up automated QA checks for common localization issues.
- Establish monitoring for translation consistency and set up automated testing procedures.

Proposal

It's important to emphasize that while this document touches on the tool's features and it's core functionality will be briefly discussed to provide context, the primary focus is on the deployment architecture and infrastructure implementation within our GitLab environment. This

approach ensures a clear understanding of how the tool and the translation pipeline fits into and operates within our broader infrastructure ecosystem.

The proposal is divided into two sections. Section 1: outlines the overall architectural flow and operational mechanics of the translation pipeline within the tool itself. Section 2: details the deployment architecture, infrastructure requirements, and implementation strategy for the proposed solution.

Section 1 : The Tool

1. Continuous monitoring and tracking: The integration tool monitors GitLab repositories and detects closed Merge Request(s) on that repositories.
2. Source file identification: If there's a change in the source en-US folder as a part of a merged MR, the tool identifies it and extracts the content from the GitLab API.
3. Translation request: Once extracted, it automatically creates an Argo Request for identified files requiring translation.
4. Vendor pipeline: Argo orchestrates translation workflow through configured vendor integrations based on GitLab's localization specifications.
5. Translated file integration to GitLab: Once the translation is complete and available to Argo, it automatically commits translated files back to GitLab.
6. Verification, review & merge: A Localization Engineer reviews the Argo's commit containing translated files, approves, and merges back to the repository.

Section 2: Deployment architecture

1. The integration tool is located in a GitLab repository, and get's build into a docker file container using GitLab's CI/CD pipeline.
2. Once that's done, the docker container will be deployed with Runway.
3. Since Runway doesn't support PostGresSQL as of now, we'll connect to a database server on Google Cloud.

Section 3: Target repositories

As of now, this tool is intended only to use with few public projects under tech-docs and marketing. Here's the complete list:

For Marketing

Repository Link	Repository Content	Branch Specifications
---	---	---
about-gitlab-com	Contents	using a translation branch

For Tech-docs

We'll be initially using a fork of the tech-docs repos, which includes all the test and prod repos located here : <https://gitlab.com/gitlab-com/localization/tech-docs-forked-projects>

In near future, once approved, we'll move to using a translation branch. Here's the list of all the repositories:

Repository Link	Current Forked Repo
gitlab-org/gitlab	localization/tech-docs-forked-projects/prod/gitlab
gitlab-org/gitlab-runner	localization/tech-docs-forked-projects/prod/gitlab-runner
gitlab-org/omnibus-gitlab	localization/tech-docs-forked-projects/prod/omnibus-gitlab
gitlab-org/omnibus-gitlab	localization/tech-docs-forked-projects/prod/omnibus-gitlab
gitlab-org/cloud-native/gitlab-operator	localization/tech-docs-forked-projects/prod/gitlab-operator
gitlab-org/gitlab-docs-hugo	localization/tech-docs-forked-projects/prod/gitlab-docs-hugo

Section 4: Database Configuration

It should be noted here that the database will never contain any sensitive data. No secrets, tokens or api keys will be stored in the database. We'll be using a separate GitLab account with limited access to the target repositories.

Data saved in Database

- Entire webhook body contents, which contains all sorts of metadata related to the GitLab MR and the repo it resides in. You can see all the data [here](#).
- When the webhooks were received and processed, along with their status regarding their processing,
- MRs identifiers and their relation to each other (what Translation MRs are tied to which Original MRs)

Possible future updates to the Database

- We will likely expand the tool for a cadence based translation instead where the application doing a periodic check and not relying on events - thus expanding on the number of events stored,
- Currently an engineer verifies the MRs before merging them to the target repository. If we add features on the tool to likely expand on merge automation, we may likely include more logging information to troubleshoot success/failures.

Note: Any data stored in the database in Phase 1 (see below) will be deleted from Spartan's environment, once the infrastructure moves to Phase 2 (see below) under GitLab.

Design and implementation details

Spartan Software, Inc. ([Spartan](#)) has helped us building a Java application that act as the intermediary tool to communicate with the Request Management System - Argo.

In summary, here's what the tool does:

- The integration tool continuously monitors certain GitLab projects or repositories,
- Anytime a Merge Request is closed on the monitored project, the integration tool receives the Merge Request data from GitLab API - that gets extracted to identify the list of en-US source language files,
- The tool then creates an Argo Request for these files that requires translation,
- Argo manages the translation on its end with the translation vendors assigned by GitLab,

- Whenever translation has completed on file(s) associated in the Request, Argo attempts to commit the file(s) back to GitLab.
- An engineer reviews the commits of the Merge Request created by Argo, approves them, and get the files merged to the repository.

Overall view of the API interactions

And the scope of actions

- Read contents of repository, including every branch and MRs
- Create new branches and add new commits to existing branches that it creates
- Create new commits
- Create MRs from that repository to itself or to other repositories

As we've noted earlier, separate GitLab account with limited access will be associated with this tool only. A user token will be created with the following permissions only with limited expiration dates:

- api
- readrepository
- writerepository

How do we authenticate the user?

For the GitLab's webhooks, we use access tokens for this user to authenticate.

For Argo, the integration is protected by hosting them on the same instance or within a firewall, thereby preventing outside access. In addition to this there's currently work-in-progress of implementing authentication Argo<->GitLab integrations. This will require enhancing Argo to also use a token when communicating with the GitLab Integration, and also the GitLab Integration being enhanced to use said token.

Lifecycle of an event

The application is event driven. The application remains dormant until either GitLab or Argo send in an HTTP Request to the application. The application will then handle the event and perform, sometimes quite complex, operations.

High Level Architecture

Event preprocessing

Quick checks and handling for large volumes of HTTP Requests before putting them into the queue.

1. Argo or GitLab events are received to ArgoController or GitLabController respectively, auth is checked, then passed unaltered to the next service.

- Purpose: Simple endpoint that allows the internal services to be interfaced with, and authenti